IDS COLLEGES
INFOTECH DEVELOPMENT SYSTEMS COLLEGES, INC
Ligao City

IDSC SUPPORTS THE SUSTAINABLE DEVELOPMENT GOALS

HANDS-ON HANDOUT: DOCKER CONTAINERIZATION

1st Semester, AY 2026-2027

Course Code/Title:	IPT 102 / Integrative Programming and Technologies 2	**Program/ Section:**	
Name of Student:		**Date:**	
Faculty:	Jeremy Zion L. Jamer	**Score:**	

GENERAL INSTRUCTIONS

- Do the steps in order. Do not skip a step, because each step uses the work from the step before.
- Windows students use **PowerShell**. Mac and Linux students use **Terminal**. You can also use the terminal inside Visual Studio Code.
- Type each command exactly as written, then press **Enter**. Capital letters, spaces, and dashes are important.
- After each step, read **You should see**. If your result is different, use the troubleshooting table before you ask for help.
- Answer each **Reflection** in your own words. Short and honest answers (1 to 3 sentences) are enough.

BEFORE YOU START

In this handout you will build a small web app with a **React (Vite)** frontend and a **.NET 10 Web API**. You will put each part in a **Docker container**. Then you will run them together with **Docker Compose** and add a **PostgreSQL** database with a **volume**. At the end, your project folder will look like this:

```
myapp/
  api/                  (.NET 10 Web API, with Dockerfile)
  web/                  (React + Vite app, with Dockerfile)
  docker-compose.yml
  .env
```

Words you will see

Word	Simple meaning
Terminal	A window where you type commands. Windows: PowerShell. Mac: Terminal.
Command	A line of text you type in the terminal. You press Enter to run it.
Port	A number that works like a door on your computer. Example: 5000.
localhost	This means "my own computer".
Image	A read-only blueprint of your app and everything it needs.
Container	A running copy of an image.
Compose	A tool that starts many containers together from one file.

PART A. GET READY

STEP 1. INSTALL DOCKER DESKTOP

Docker Desktop is the program that runs containers on your computer. It gives you the **`docker`** command.

What to do

1. Check your computer. You need Windows 10 (version 22H2 or newer) or Windows 11, macOS, or Linux. You also need at least 4 GB of RAM (8 GB is better) and about 10 GB of free disk space.
2. Open your browser and go to **`docker.com`**. Download **Docker Desktop** for your system.
3. Run the installer. On Windows, keep the option **Use WSL 2 instead of Hyper-V** ticked. Restart your computer when the installer asks you to.
4. Open **Docker Desktop**. Accept the terms. You can skip the sign-in. Wait until the bottom-left corner says **Engine running**.
5. Open a terminal (PowerShell on Windows, Terminal on Mac) and type these two commands. Press Enter after each one.

```
docker --version
docker run hello-world
```

You should see

The first command shows a version number, for example **`Docker version 28.x.x`**. The second command downloads a tiny image and prints **Hello from Docker!**. This means Docker works.

If something goes wrong

Problem or error message	What to do
Docker Desktop says virtualization is not enabled	Virtualization is turned off in your computer BIOS. Restart the computer, open the BIOS (often F2, F10, or Del), and turn on **Intel VT-x** or **AMD SVM**. Search your laptop model + "enable virtualization" if you are not sure.
WSL 2 installation is incomplete	Open PowerShell as Administrator and run **`wsl --update`**. Restart the computer and open Docker Desktop again.
'docker' is not recognized as an internal or external command	Close the terminal and open a new one. If it still fails, restart the computer. Check that Docker Desktop is installed and open.
error during connect, or Cannot connect to the Docker daemon	Docker Desktop is not running. Open it and wait for **Engine running** before you type commands.
The computer becomes very slow	Close other programs and browser tabs. Docker needs memory. Ask your instructor if your computer has less than 4 GB of RAM.

Reflection

1. In your own words, what is Docker? What problem does it solve?

2. Did you have any problem during the installation? Write the problem and how you fixed it. If you had none, write how you checked that Docker works.

STEP 2. INSTALL THE OTHER TOOLS

You will build the app on your computer first. Then you will move it into containers. For this you need three tools.

What to do

1. Install **Visual Studio Code** (code editor) from **`code.visualstudio.com`**.
2. Install **Node.js** (choose the **LTS** version) from **`nodejs.org`**. This is for the React app.
3. Install the **.NET 10 SDK** from **`dotnet.microsoft.com/download`**. Choose the **SDK**, not the Runtime. This is for the Web API.
4. Close and open a new terminal. Check that all tools work:

```
node --version
npm --version
dotnet --version
```

You should see

`node --version` shows **v22** or higher (at least v20.19). **`dotnet --version`** shows a number that starts with **10**.

If something goes wrong

Problem or error message	What to do
'node' or 'dotnet' is not recognized	Close the terminal and open a new one. If it still fails, restart the computer. If it still fails, install the tool again.
`dotnet --version` shows 8 or 9	You have an older SDK. Install the **.NET 10 SDK** from the website and check again.
npm.ps1 cannot be loaded because running scripts is disabled	In PowerShell, run **`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`** and answer **Y**. Open a new terminal.
Node version is too old for Vite	Vite needs Node 20.19 or 22.12 and newer. Install the newest **LTS** version from nodejs.org.

Reflection

1. If we will run the app in containers later, why do we first install Node.js and .NET on our own computer?

2. Write the versions you installed. Node.js: _____ .NET: _____ Docker: _____

PART B. CREATE THE WEB APP

STEP 3. CREATE THE .NET 10 WEB API

The API is the backend. It sends a message to the frontend. You will make one endpoint: **`/api/message`**.

What to do

1. Create a project folder and open it in VS Code:

```
mkdir myapp
cd myapp
code .
```

2. In VS Code, open a terminal (**Terminal > New Terminal**). Create the API project:

```
dotnet new web -n api
```

3. Open **`api/Program.cs`**. Delete everything inside and paste this code. Save the file.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
        policy.WithOrigins(
                "http://localhost:3000",
                "http://localhost:5173")
              .AllowAnyHeader()
              .AllowAnyMethod());
});

var app = builder.Build();
app.UseCors();

app.MapGet("/api/message", (IConfiguration config) =>
{
    var text = config["Message"] ?? "Hello from the API";
    return Results.Ok(new { message = text });
});

app.Run();
```

Note: The code does three things. It allows the React app to call the API (CORS). It reads a setting called **`Message`**. It returns the message as JSON at **`/api/message`**.

4. Run the API:

```
cd api
dotnet run --urls http://localhost:5000
```

5. Open your browser and go to **`http://localhost:5000/api/message`**. When you finish, go back to the terminal and press **Ctrl + C** to stop the API.

You should see

The browser shows: **`{"message":"Hello from the API"}`**. The terminal shows **Now listening on: http://localhost:5000**.

If something goes wrong

Problem or error message	What to do
Address already in use, or port 5000 is busy	Another program uses port 5000. On Mac, turn off **AirPlay Receiver** in System Settings. Or use port **5050** instead of 5000 in this whole handout.
Couldn't find a project to run	You are in the wrong folder. Type **`cd api`** first. Use **`dir`** (Windows) or **`ls`** (Mac) and check that you see **`api.csproj`**.
Build errors or red lines in Program.cs	Check that you pasted all the code. Check every bracket and semicolon. Compare your file with the handout line by line.
This site can't be reached	The API is not running. Look at the terminal and check that it says **Now listening**. Check that the address has no typing mistake.
dotnet is not recognized	Go back to Step 2 and install the .NET 10 SDK. Open a new terminal.

Reflection

1. In your own words, what does the endpoint **`/api/message`** do?

2. Why did we add CORS to the code? What can happen if we do not add it?

STEP 4. CREATE THE REACT (VITE) FRONTEND

The frontend is what the user sees in the browser. It will call your API and show the message.

What to do

1. Open a **new terminal** (click the + button). Make sure you are in the **`myapp`** folder, not inside **`api`**. Create the React project:

```
npm create vite@latest web -- --template react
```

   Note: If it asks questions, choose **No** for each one.

2. Install the packages:

```
cd web
npm install
```

3. Open **`web/src/App.jsx`**. Delete everything inside and paste this code. Save the file.

```
import { useEffect, useState } from "react";

const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000";

function App() {
  const [message, setMessage] = useState("Loading...");

  useEffect(() => {
    fetch(`${API_URL}/api/message`)
      .then((res) => res.json())
      .then((data) => setMessage(data.message))
      .catch(() => setMessage("Cannot reach the API"));
  }, []);

  return (
    <div style={{ padding: 32, fontFamily: "Arial" }}>
      <h1>My Docker App</h1>
      <p>{message}</p>
    </div>
  );
}

export default App;
```

4. Start the API in the first terminal (**`cd api`** then **`dotnet run --urls http://localhost:5000`**). In the second terminal (inside **`web`**) start React:

```
npm run dev
```

5. Open **`http://localhost:5173`** in your browser. When you finish, press **Ctrl + C** in both terminals.

You should see

A page with the heading **My Docker App** and the text **Hello from the API**.

If something goes wrong

Problem or error message	What to do
The page says Cannot reach the API	The API is not running, or the port is wrong. Check the first terminal. Press **F12** in the browser and read the red errors in the Console.
Console shows: blocked by CORS policy	Check the CORS lines in **`Program.cs`**. The address **`http://localhost:5173`** must be in the list. Stop the API (Ctrl + C) and start it again after you change the code.
The page is blank (white)	Press F12 and read the Console error. Check that you pasted all of **`App.jsx`** and saved the file.
Port 5173 is in use	Another Vite app is open. Close the other terminal that runs Vite. The API only allows port 5173.
npm is not recognized, or scripts are disabled	See the fixes in Step 2.

Reflection

1. What is the job of the frontend? What is the job of the API? How do they talk to each other?

2. Right now your app only works if Node.js and .NET are installed. What problem can happen when you give this app to a classmate?

PART C. PUT EACH PART IN A CONTAINER

STEP 5. CONTAINERIZE THE API

You will write a Dockerfile. A Dockerfile is a text file with the steps to build an image. Then you will run a container from that image.

What to do

1. Create a file named **`api/.dockerignore`** with this content:

```
bin
obj
.vs
```

2. Create a file named **`api/Dockerfile`**. The name has no extension. Paste this code:

```
# Stage 1: build and publish the API
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish --no-restore

# Stage 2: run the API
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "api.dll"]
```

3. Go back to the **`myapp`** folder in the terminal. Build the image:

```
docker build -t myapp-api:1.0 ./api
```

4. Run a container from the image:

```
docker run -d -p 5000:8080 --name myapp-api myapp-api:1.0
```

Note: **`-d`** runs it in the background. **`-p 5000:8080`** connects port 5000 on your computer to port 8080 in the container. **`--name`** gives the container a name.

5. Check it. Then open **`http://localhost:5000/api/message`** in the browser.

```
docker ps
docker logs myapp-api
```

6. Stop and remove the container:

```
docker stop myapp-api
docker rm myapp-api
```

You should see

`docker ps` shows **`myapp-api`** with the status **Up**. The browser shows the same JSON message as in Step 3.

If something goes wrong

Problem or error message	What to do
failed to read dockerfile, or no such file: Dockerfile	The file name must be exactly **`Dockerfile`** (capital D, no .txt). On Windows, turn on **File name extensions** in File Explorer to check.
MSB1003: Specify a project or solution file	Docker cannot find the .csproj file. Build from the **`myapp`** folder with **`./api`** at the end, and check that **`Dockerfile`** is inside the **`api`** folder.
port is already allocated, or Bind for 0.0.0.0:5000 failed	Something else uses port 5000. Close any terminal that still runs **`dotnet run`**. Run **`docker ps`** and stop old containers with **`docker stop NAME`**.

Problem or error message	What to do
The container name is already in use	Remove the old one: **`docker rm -f myapp-api`**. Then run the container again.
The container stops right away (Exited)	Run **`docker logs myapp-api`** and read the error. Check that **`api.dll`** in the last line of the Dockerfile matches your project name.
Browser cannot open the page but the container is Up	Check the port mapping. It must be **`5000:8080`** (computer:container). Read **`docker logs myapp-api`**.
error during connect	Docker Desktop is not running. Open it first.

Reflection

1. What is the difference between an **image** and a **container**? Give your own example.

2. Your Dockerfile has two **`FROM`** lines (two stages). Why do we use two stages?

STEP 6. CONTAINERIZE THE REACT APP

The React app also needs a Dockerfile. Vite builds the app into plain files. Nginx (a web server) will serve those files.

What to do

1. Create **`web/.dockerignore`** with this content:

```
node_modules
dist
.git
```

2. Create **`web/Dockerfile`** and paste this code:

```
# Stage 1: build the React app
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=http://localhost:5000
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

# Stage 2: serve the files with Nginx
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

Note: Vite reads **`VITE_API_URL`** when it **builds**, not when the container runs. That is why the Dockerfile has **`ARG`** and **`ENV`** before **`npm run build`**.

3. Build the image from the **`myapp`** folder:

```
docker build -t myapp-web:1.0 ./web
```

4. Start both containers:

```
docker run -d -p 5000:8080 --name myapp-api myapp-api:1.0
docker run -d -p 3000:80 --name myapp-web myapp-web:1.0
```

5. Open **`http://localhost:3000`** in the browser. Then stop and remove both containers:

```
docker stop myapp-web myapp-api
docker rm myapp-web myapp-api
```

You should see

The same page as before: **My Docker App** and **Hello from the API**. This time both parts run inside containers.

If something goes wrong

Problem or error message	What to do
npm ci can only install with an existing package-lock.json	The lock file is missing. Run **`npm install`** inside the **`web`** folder (Step 4), then build again.
The page says Cannot reach the API	The API container is not running. Run **`docker ps`**. Start it again with the first **`docker run`** command above.
Port 3000 is already allocated	Another program uses port 3000. Close it, or stop old containers with **`docker ps`** and **`docker stop NAME`**.
The page shows Welcome to nginx!	Your files were not copied. Check that the **`COPY --from=build`** line is correct and that **`npm run build`** makes a **`dist`** folder.
I changed my code but the page did not change	The image still has the old code. Build the image again, then remove and run a new container.
The first build is very slow	This is normal. Docker downloads the base images the first time. The next builds are faster.

Reflection

1. Why do we copy only the **`dist`** folder into the Nginx stage and not the whole project?

2. Type **`docker images`**. Write the names and sizes of your two images. Which one is bigger? Why do you think so?

PART D. RUN EVERYTHING TOGETHER

STEP 7. CREATE docker-compose.yml

Starting containers one by one is slow. Docker Compose starts everything with one command.

What to do

1. In the **`myapp`** folder (beside **`api`** and **`web`**), create a file named **`docker-compose.yml`**. Paste this:

```
services:
  api:
    build: ./api
    ports:
      - "5000:8080"

  web:
    build: ./web
    ports:
      - "3000:80"
    depends_on:
      - api
```

Note: Use **spaces**, never the Tab key. Indentation (the spaces at the start of a line) is important in YAML files.

2. Build and start everything:

```
docker compose up --build
```

3. Open **`http://localhost:3000`**. When you finish, press **Ctrl + C** and then type:

```
docker compose down
```

You should see

The terminal shows logs from both **`api`** and **`web`**. The browser shows **Hello from the API**. Tip: **`docker compose up -d`** runs in the background, and **`docker compose ps`** lists your services.

If something goes wrong

Problem or error message	What to do
yaml: line N: mapping values are not allowed, or found character that cannot start any token	The YAML spacing is wrong. Look at line N. Remove any Tab characters. Use 2 spaces for each level and align the lines like the handout.
no configuration file provided: not found	You are in the wrong folder. Use **`cd`** to go to the **`myapp`** folder. The file must be named **`docker-compose.yml`**.
port is already allocated	Containers from Step 6 are still running. Run **`docker ps`** and stop them with **`docker stop NAME`**.
'docker-compose' is not recognized	Use a space, not a dash: **`docker compose`**.
Warning: the attribute version is obsolete	Ignore it, or delete the **`version`** line. It is not needed.
Service web failed to build	Read the red error. It names the line that failed. Fix that Dockerfile and run **`docker compose up --build`** again.

Reflection

1. Write three things Docker Compose does for us that are harder with many **`docker run`** commands.

2. What does **`depends_on`** do? What does it **not** do?

STEP 8. ENVIRONMENT VARIABLES AND THE .env FILE

An environment variable is a setting that you give to a container from outside. You can change the setting without changing the code.

What to do

1. In the **`myapp`** folder, create a file named **`.env`** (no spaces around the equal sign):

```
APP_MESSAGE=Hello from Docker Compose!
DB_PASSWORD=change_me_123
```

2. Replace your **`docker-compose.yml`** with this version:

```
services:
  api:
    build: ./api
    ports:
      - "5000:8080"
    environment:
      Message: ${APP_MESSAGE}

  web:
    build:
      context: ./web
      args:
        VITE_API_URL: http://localhost:5000
    ports:
      - "3000:80"
    depends_on:
      - api
```

3. Start it again, then refresh **`http://localhost:3000`**:

```
docker compose up -d --build
```

4. Now change the message. Open **`.env`**, write your own message, and save. Then run this and refresh the page:

```
docker compose up -d
```

Note: In .NET, a setting such as **`ConnectionStrings:Default`** is written with double underscores as an environment variable: **`ConnectionStrings__Default`**. If you use Git, add **`.env`** to your **`.gitignore`** file. Never upload passwords to GitHub.

You should see

The page shows **Hello from Docker Compose!** first, and then your own message after you change **`.env`**.

If something goes wrong

Problem or error message	What to do
The page still shows the old message	Check that you saved **`.env`** and ran **`docker compose up -d`** again. Press **Ctrl + F5** in the browser.
The APP_MESSAGE variable is not set. Defaulting to a blank string	Compose cannot find **`.env`**. The file must be in the same folder as **`docker-compose.yml`**. Check that the name is not **`.env.txt`**.
services.api.environment must be a mapping	The YAML spacing is wrong under **`environment:`**. Use 2 more spaces than **`environment`** and no Tab.
The message looks wrong or has strange spaces	Do not put spaces around the **=** in **`.env`**. Write **`APP_MESSAGE=Hello`**, not **`APP_MESSAGE = Hello`**.

Reflection

1. Why do we put values like passwords in a **`.env`** file and not directly in **`docker-compose.yml`** or the code?

2. How do you write the setting **`ConnectionStrings:Default`** as an environment variable in .NET?

STEP 9. NETWORKING AND THE DATABASE

Compose puts all services in one private network. Containers can find each other by **service name**. You will add a PostgreSQL database to see this.

What to do

1. Open **`docker-compose.yml`**. Add the **`db`** service at the end of the file, under **`services`** (same spacing as **`api`** and **`web`**):

```
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: appdb
```

   Note: The **`db`** service has no **`ports`**. This means your computer cannot reach it directly. Only the other containers can.

2. Start everything and check the three services:

```
docker compose up -d --build
docker compose ps
docker network ls
```

3. Test the network. The web container can reach the other containers by name:

```
docker compose exec web ping -c 2 api
docker compose exec web ping -c 2 db
```

4. Open the database from inside its container. Type **`\l`** to list databases and **`\q`** to quit:

```
docker compose exec db psql -U postgres -d appdb
```

You should see

`docker compose ps` shows **api**, **web**, and **db** as running. A network named **`myapp_default`** is in the list. The ping commands show replies (for example **2 packets received**).

If something goes wrong

Problem or error message	What to do
ping: bad address 'api'	The services are not running, or you are in the wrong folder. Go to the **`myapp`** folder and run **`docker compose ps`**.
service "web" is not running	Start the services first with **`docker compose up -d`**. Use **`docker compose logs web`** to see the error.
The db container keeps stopping, or says superuser password is not specified	The **`DB_PASSWORD`** line is missing in **`.env`**. Add it, save, and run **`docker compose up -d`** again.
psql: role does not exist	The default user is **`postgres`**. Use **`-U postgres`** exactly as shown.
the input device is not a TTY (Git Bash on Windows)	Use PowerShell or the VS Code terminal instead of Git Bash.

Reflection

1. How does the web container find the API container? Why does the database have no **`ports`** line?

2. What does **localhost** mean inside a container? Why is this a common mistake when connecting to a database?

STEP 10. VOLUMES: KEEP YOUR DATA

A container forgets its data when it is removed. A **volume** is storage outside the container. The data stays even if the container is deleted.

What to do

1. Update the **db** service and add a **volumes** part at the **bottom** of the file. **volumes:** at the bottom must start at the left edge, like **services:**.

```
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: appdb
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

2. Apply the change. Then create a table and add one row:

```
docker compose up -d
docker compose exec db psql -U postgres -d appdb -c "CREATE TABLE students (id SERIAL
PRIMARY KEY, name TEXT);"
docker compose exec db psql -U postgres -d appdb -c "INSERT INTO students (name) VALUES
('Juan');"
docker compose exec db psql -U postgres -d appdb -c "SELECT * FROM students;"
```

3. **Test 1.** Remove the containers (not the volume). Start again and read the data:

```
docker compose down
docker volume ls
docker compose up -d
docker compose exec db psql -U postgres -d appdb -c "SELECT * FROM students;"
```

4. **Test 2.** Now delete the volume too (**-v**). Start again and read the data:

```
docker compose down -v
docker compose up -d
docker compose exec db psql -U postgres -d appdb -c "SELECT * FROM students;"
```

You should see

After Test 1, the row **Juan** is still there. **docker volume ls** shows **myapp_db-data**. After Test 2, you see an error: **relation "students" does not exist**. The data is gone because you deleted the volume.

If something goes wrong

Problem or error message	What to do
relation "students" does not exist (after Test 1)	The volume is missing or you used **down -v**. Check the **volumes** lines in your file for typing mistakes. Then create the table and row again.
password authentication failed	You changed **DB_PASSWORD** after the volume was created. Postgres keeps the old password. Run **docker compose down -v** and start again (this deletes the data).
connection refused, or the database system is starting up	The database needs a few seconds to start. Wait 10 seconds and try again.
volume is in use, cannot remove volume	Run **docker compose down** first. Then remove it with **docker volume rm myapp_db-data**.
YAML error near volumes:	The **volumes:** at the bottom must not have spaces before it. The line **db-data:** under it has 2 spaces.

Reflection

1. What happened to your data after **docker compose down**? What happened after **docker compose down -v**? Explain why.

2. Think of a real system (for example a school enrollment system). Why is losing data a serious problem? How does a volume help?

PART E. FINISH

STEP 11. CLEAN UP AND REVIEW

Docker keeps old containers, images, and volumes. Clean them up so your disk does not become full.

What to do

1. Stop everything and delete the volume and the images that Compose built:

```
docker compose down -v --rmi local
```

2. Delete the two images you built by hand in Steps 5 and 6:

```
docker rmi myapp-api:1.0 myapp-web:1.0
```

3. Check that things are gone:

```
docker ps -a
docker images
docker volume ls
```

You should see

The lists show none of your **myapp** containers, images, or volumes. Your source code in the **myapp** folder is still safe. You can build everything again with **`docker compose up --build`**.

If something goes wrong

Problem or error message	What to do
image is being used by stopped container	Run **`docker ps -a`**. Remove the old container with **`docker rm NAME`**. Then remove the image again.
No such image	It is already deleted. You can ignore this message.
Docker uses too much disk space	Run **`docker system prune`**. It removes stopped containers and unused images. Read the question before you press **y**.

Reflection

1. List the three most important things you learned in this handout.

2. Which step was the hardest for you? What made it hard, and how did you solve it?

3. How can you use Docker in your Integrative Programming 2 finals project?

COMMAND CHEAT SHEET

Command	What it does
docker build -t NAME:TAG ./folder	Build an image from a Dockerfile
docker run -d -p HOST:CONTAINER --name NAME IMAGE	Start a container in the background
docker ps / docker ps -a	List running containers / all containers
docker logs NAME	Show the output of a container
docker stop NAME / docker rm NAME	Stop a container / remove a container
docker images / docker rmi IMAGE	List images / remove an image
docker compose up -d --build	Build and start all services in the background
docker compose down / down -v	Stop and remove services / also delete volumes
docker compose logs -f SERVICE	Follow the logs of one service
docker compose exec SERVICE COMMAND	Run a command inside a running service
docker volume ls / docker network ls	List volumes / list networks

***** END OF HANDOUT *****